Date:15/10/26

CO1-AT3 CODING CHALLENGE

COURSE CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REG.NO: 192324281 **NAME: MONIKA R**

Question 1: Intelligent Email and Password Validator using Regular Expressions

Algorithm

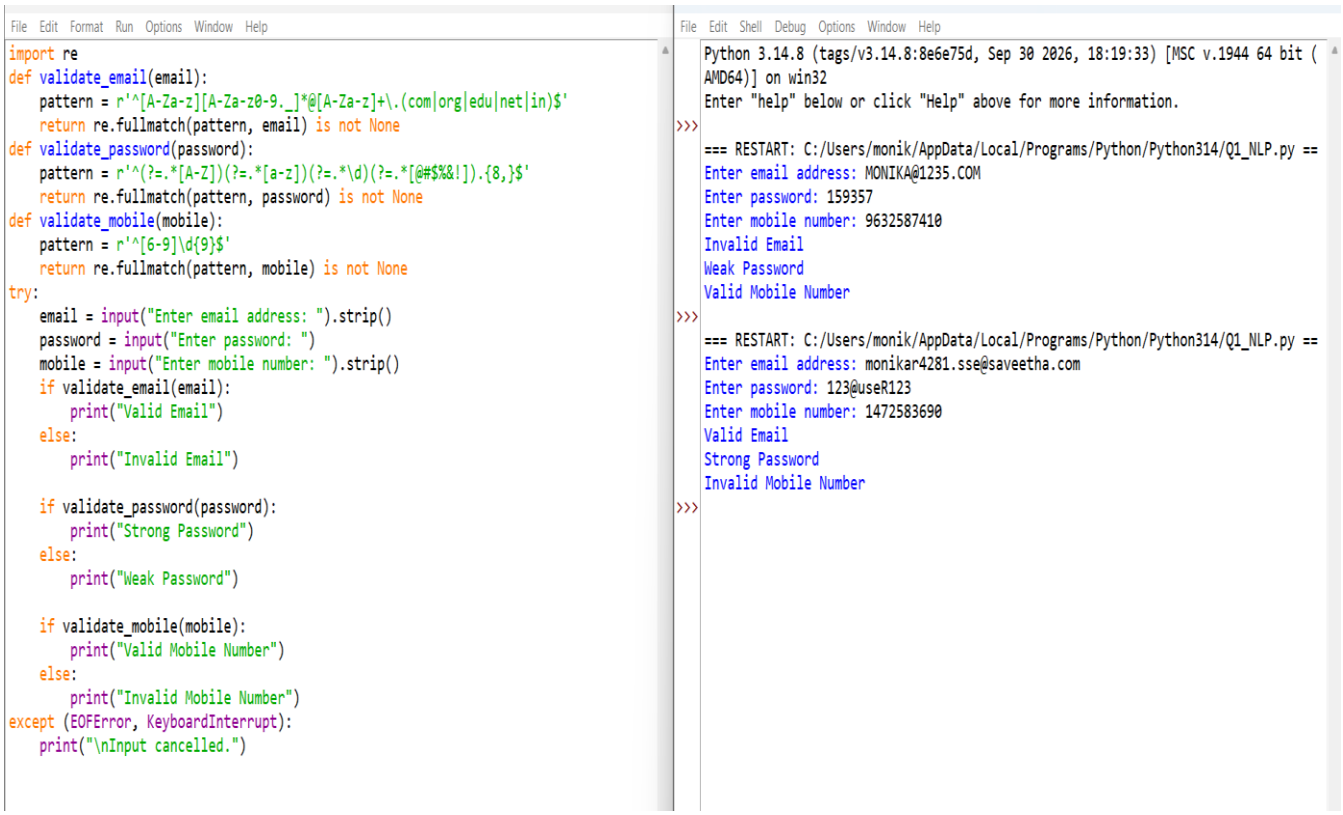
1. Import the re module.
2. Read the email address, password, and mobile number from the user.
3. Use a regular expression to check whether the email:
    - Starts with an alphabet.
    - Contains only letters, digits, dots, and underscores before @.
    - Has an alphabetic domain.
    - Ends with .com, .org, .edu, .net, or .in.
4. Check whether the password:
    - Contains at least 8 characters.
    - Has an uppercase letter and a lowercase letter.
    - Has at least one digit.
    - Has at least one permitted special character: @ # $ % & !.
5. Check whether the mobile number contains exactly 10 digits and starts with a digit from 6 to 9.
6. Display the appropriate validation result for all three inputs.

CODE:

```
import re
def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return re.fullmatch(pattern, email) is not None
def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
    return re.fullmatch(pattern, password) is not None
def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    return re.fullmatch(pattern, mobile) is not None
try:
```

```
    email = input("Enter email address: ").strip()
    password = input("Enter password: ")
    mobile = input("Enter mobile number: ").strip()
    if validate_email(email):
        print("Valid Email")
    else:
        print("Invalid Email")
    if validate_password(password):
        print("Strong Password")
    else:
        print("Weak Password")
    if validate_mobile(mobile):
        print("Valid Mobile Number")
    else:
        print("Invalid Mobile Number")
except (EOFError, KeyboardInterrupt):
    print("\nInput cancelled.")
```

OUTPUT:



```
File  Edit  Format  Run  Options  Window  Help
import re
def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return re.fullmatch(pattern, email) is not None
def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
    return re.fullmatch(pattern, password) is not None
def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    return re.fullmatch(pattern, mobile) is not None
try:
    email = input("Enter email address: ").strip()
    password = input("Enter password: ")
    mobile = input("Enter mobile number: ").strip()
    if validate_email(email):
        print("Valid Email")
    else:
        print("Invalid Email")

    if validate_password(password):
        print("Strong Password")
    else:
        print("Weak Password")

    if validate_mobile(mobile):
        print("Valid Mobile Number")
    else:
        print("Invalid Mobile Number")
except (EOFError, KeyboardInterrupt):
    print("\nInput cancelled.")
```

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:33) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/Q1_NLP.py ==
Enter email address: MONIKA@1235.COM
Enter password: 159357
Enter mobile number: 9632587410
Invalid Email
Weak Password
Valid Mobile Number
>>>
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/Q1_NLP.py ==
Enter email address: monikar4281.sse@saveetha.com
Enter password: 123@useR123
Enter mobile number: 1472583690
Valid Email
Strong Password
Invalid Mobile Number
>>>
```

Question 2: Design a Finite State Automata (FSA) Simulator

Algorithm

1. Define the DFA states, input alphabet, transition table, initial state, and final states.
2. Read the number of input strings to test.
3. For each string, begin at the initial state.
4. Read each character and use the transition table to determine the next state.
5. Record each visited state in the transition path.
6. If a character is outside the input alphabet or a transition is missing, reject the string.
7. After processing the entire string, check whether the current state is a final state.
8. Display the transition path and the result.

DFA design: Strings ending with ab

The question paper provides an example language consisting of strings ending with ab, with input abaab.

We can use the following DFA over the alphabet {a, b}.

- q0: Initial state; no relevant ending has been matched.
- q1: The latest character is a.
- q2: The string ends with ab; this is the accepting state.

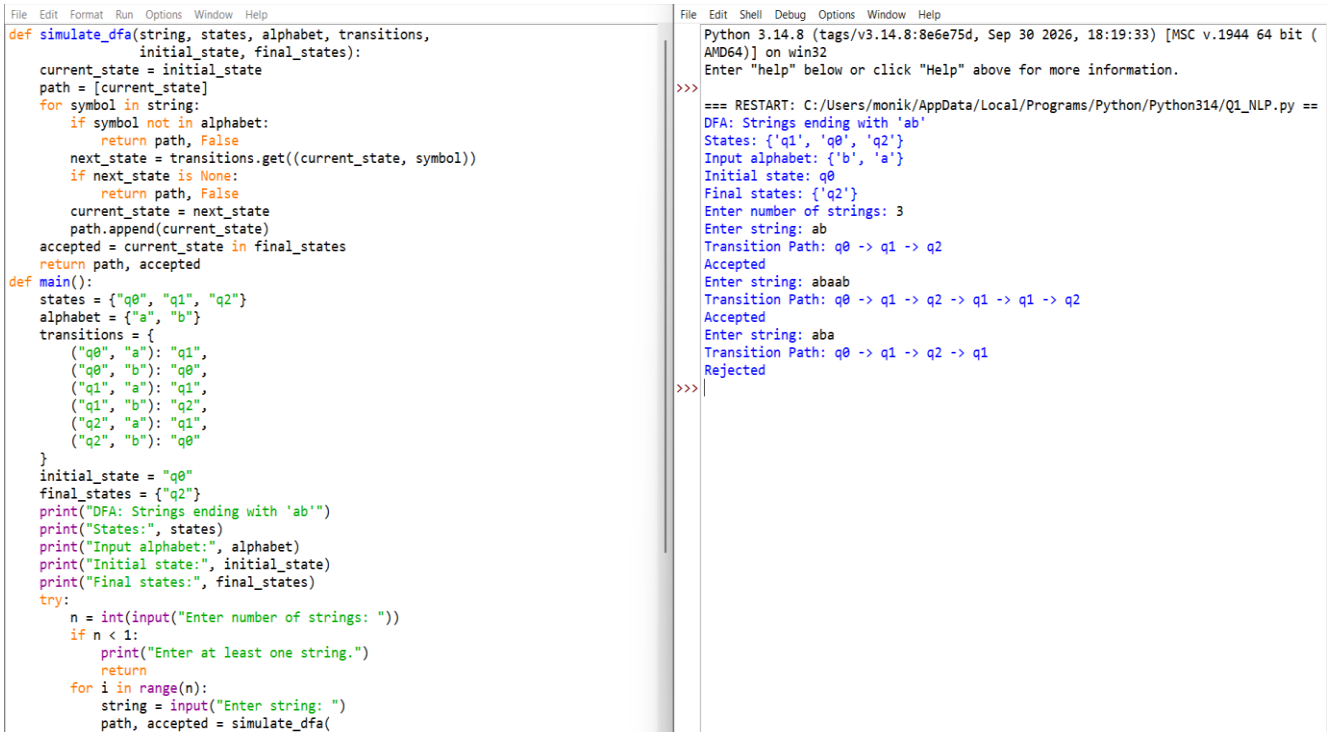
CODE:

```
def simulate_dfa(string, states, alphabet, transitions,
                 initial_state, final_states):
    current_state = initial_state
    path = [current_state]
    for symbol in string:
        if symbol not in alphabet:
            return path, False
        next_state = transitions.get((current_state, symbol))
        if next_state is None:
            return path, False
        current_state = next_state
        path.append(current_state)
    accepted = current_state in final_states
    return path, accepted
def main():
    states = {"q0", "q1", "q2"}
    alphabet = {"a", "b"}
```

```
    transitions = {
        ("q0", "a"): "q1",
        ("q0", "b"): "q0",
        ("q1", "a"): "q1",
        ("q1", "b"): "q2",
        ("q2", "a"): "q1",
        ("q2", "b"): "q0"
    }
    initial_state = "q0"
    final_states = {"q2"}
    print("DFA: Strings ending with 'ab'")
    print("States:", states)
    print("Input alphabet:", alphabet)
    print("Initial state:", initial_state)
    print("Final states:", final_states)
    try:
        n = int(input("Enter number of strings: "))
        if n < 1:
            print("Enter at least one string.")
            return
        for i in range(n):
            string = input("Enter string: ")
            path, accepted = simulate_dfa(
                string, states, alphabet, transitions,
                initial_state, final_states
            )
            print("Transition Path:", " -> ".join(path))
            if accepted:
                print("Accepted")
            else:
                print("Rejected")
    except ValueError:
        print("Invalid input. Enter an integer for the count.")

if __name__ == "__main__":
    main()
```

OUTPUT:



```
File  Edit  Format  Run  Options  Window  Help
def simulate_dfa(string, states, alphabet, transitions,
                 initial_state, final_states):
    current_state = initial_state
    path = [current_state]
    for symbol in string:
        if symbol not in alphabet:
            return path, False
        next_state = transitions.get((current_state, symbol))
        if next_state is None:
            return path, False
        current_state = next_state
        path.append(current_state)
    accepted = current_state in final_states
    return path, accepted
def main():
    states = {"q0", "q1", "q2"}
    alphabet = {"a", "b"}
    transitions = {
        ("q0", "a"): "q1",
        ("q0", "b"): "q0",
        ("q1", "a"): "q1",
        ("q1", "b"): "q2",
        ("q2", "a"): "q1",
        ("q2", "b"): "q0"
    }
    initial_state = "q0"
    final_states = {"q2"}
    print("DFA: Strings ending with 'ab'")
    print("States:", states)
    print("Input alphabet:", alphabet)
    print("Initial state:", initial_state)
    print("Final states:", final_states)
    try:
        n = int(input("Enter number of strings: "))
        if n < 1:
            print("Enter at least one string.")
            return
        for i in range(n):
            string = input("Enter string: ")
            path, accepted = simulate_dfa(
```

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:33) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/Q1_NLP.py ==
DFA: Strings ending with 'ab'
States: {'q1', 'q0', 'q2'}
Input alphabet: {'b', 'a'}
Initial state: q0
Final states: {'q2'}
Enter number of strings: 3
Enter string: ab
Transition Path: q0 -> q1 -> q2
Accepted
Enter string: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
Enter string: aba
Transition Path: q0 -> q1 -> q2 -> q1
Rejected
>>>
```

Question 3: Smart Pattern Matching Engine using Regular Expressions

Algorithm

1. Import the re module.
2. Read a text passage from the user.
3. Display a menu containing the available search options.
4. Read the user's menu selection.
5. Apply the appropriate regular expression:
    - Date search: Find dates in DD/MM/YYYY format.
    - Phone number search: Find 10-digit mobile numbers beginning with 6–9.
    - Hashtag search: Find hashtags such as #NLP.
    - Mention search: Find usernames such as @OpenAI.
    - Prefix search: Find words starting with the specified prefix.
    - Suffix search: Find words ending with the specified suffix.
    - Word search: Find occurrences of a specified word.
6. Display all matching patterns.
7. If no match exists, display an appropriate message.
8. Allow the user to select another search option or exit.

CODE:

```
import re
def search_pattern(text, choice):
    patterns = {
        "1": r'\b\d{2}/\d{2}/\d{4}\b',
        "2": r'(?<!\d)[6-9]\d{9}(?!\d)',
        "3": r'(?<!\w)#[A-Za-z0-9_]+',
        "4": r'(?<!\w)@[A-Za-z0-9_]+'
    }
    if choice in patterns:
        return re.findall(patterns[choice], text)
    if choice == "5":
        prefix = input("Enter prefix: ")
        if not prefix:
            return []
        pattern = r'\b' + re.escape(prefix) + r'\w*\b'
        return re.findall(pattern, text, re.IGNORECASE)
    if choice == "6":
        suffix = input("Enter suffix: ")
        if not suffix:
            return []
        pattern = r'\b\w*' + re.escape(suffix) + r'\b'
        return re.findall(pattern, text, re.IGNORECASE)
    if choice == "7":
        word = input("Enter word to search: ")
        if not word:
            return []
        pattern = r'\b' + re.escape(word) + r'\b'
        return re.findall(pattern, text, re.IGNORECASE)
    return None
def main():
    print("Enter text (multiple lines allowed).")
    print("Type END on a separate line to finish:")
    lines = []
    while True:
        line = input()
```

```
        if line == "END":
            break
        lines.append(line)
    text = "\n".join(lines)
    while True:
        print("\n--- Smart Pattern Matching Engine ---")
        print("1. Search Date")
        print("2. Search Phone Number")
        print("3. Search Hashtag")
        print("4. Search Mention")
        print("5. Search Prefix")
        print("6. Search Suffix")
        print("7. Search Word")
        print("8. Exit")
        choice = input("Enter your choice: ").strip()
        if choice == "8":
            print("Exiting search engine.")
            break
        if choice not in {"1", "2", "3", "4", "5", "6", "7"}:
            print("Invalid menu choice.")
            continue
        matches = search_pattern(text, choice)
        if matches is None:
            print("Invalid menu choice.")
        elif matches:
            print("Matching patterns:")
            for match in matches:
                print(match)
        else:
            print("No matching patterns found.")
if __name__ == "__main__":
    main()
```

OUTPUT:

```
File  Edit  Format  Run  Options  Window  Help
import re
def search_pattern(text, choice):
    patterns = {
        "1": r'\b\d{2}/\d{2}/\d{4}\b',
        "2": r'(?<!\d)[6-9]\d{9}(?!\d)',
        "3": r'(?<!\w)#[A-Za-z0-9_]+',
        "4": r'(?<!\w)@[A-Za-z0-9_]+'
    }
    if choice in patterns:
        return re.findall(patterns[choice], text)
    if choice == "5":
        prefix = input("Enter prefix: ")
        if not prefix:
            return []
        pattern = r'\b' + re.escape(prefix) + r'\w*\b'
        return re.findall(pattern, text, re.IGNORECASE)
    if choice == "6":
        suffix = input("Enter suffix: ")
        if not suffix:
            return []
        pattern = r'\b\w*' + re.escape(suffix) + r'\b'
        return re.findall(pattern, text, re.IGNORECASE)
    if choice == "7":
        word = input("Enter word to search: ")
        if not word:
            return []
        pattern = r'\b' + re.escape(word) + r'\b'
        return re.findall(pattern, text, re.IGNORECASE)
    return None
def main():
    print("Enter text (multiple lines allowed).")
    print("Type END on a separate line to finish:")
    lines = []
    while True:
        line = input()
        if line == "END":
            break
        lines.append(line)
    text = "\n".join(lines)
    while True:
```

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:33) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/Q1_NLP.py ==
Enter text (multiple lines allowed).
Type END on a separate line to finish:
Meeting on 12/09/2026
... Call 9876543210
... #NLP
... @OpenAI
... natural language processing
... END

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 1
Matching patterns:
12/09/2026

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 2
Matching patterns:
9876543210
```